AI Engineering and Claude Architect · Part I, Foundations

Generative AI and LLMs, Lesson 1

Tokens and Tokenization

You migrate to a newer Claude model, changing nothing else. Your prompt didn't grow by a single character, and yet the token count jumped and your budget alerts fired. To understand why, you need to know what a token actually is.

Module	AI02 of 13
Lesson	1 of 7
Prerequisites	AI01
Exam relevance	Tokenization specifics are out of scope. The effect on context budgets supports TS 5.1
Reading time	~30 minutes

Contents

1. Tokens and Tokenization

Lesson 1 of 7 · Generative AI and LLM Fundamentals

Tokens and Tokenization

Key idea

A token is the unit an LLM reads and writes, and also the unit you pay for and run out of. Tokens aren't words. A tokenizer splits text into pieces it learned from data, so common words become one token, while rare identifiers shatter into many. **Token counts therefore depend on both your content and the model, so measure them instead of estimating.**

> *Before: why text must become numbers (AI01 L4). Teaches: what tokens are, BPE, special tokens, why JSON costs more than prose, and token budgeting in production. Used by: AI02 L2 (embeddings), AI03 L5 (cost), AI03 L6 (context budgets), AI10 L5 (cost engineering).*

The hook: same prompt, more tokens

Your team moves a support assistant from an older Claude model to a newer one. The prompt is byte-for-byte identical. Afterwards, input tokens per request are up by about 30%.

Nobody made a mistake. Anthropic's token counting documentation explains that Claude 4.7 and later models use a newer tokenizer, and that the same input text produces "approximately 30 percent more tokens" than on earlier models. The exact increase depends on the content.

Key idea

A token count isn't a property of your text. It's a property of your text and a specific model's tokenizer. So a budget measured against one model doesn't transfer to another. The documentation says directly: recount prompts against the model you plan to use.

To see why this can happen at all, let's build a tokenizer.

Intuition: a shorthand dictionary

Imagine a court stenographer with a shorthand book. Very common words each have one symbol. Rare words get spelled out using symbols for common fragments: "un", "believ", "able". The stenographer can write anything, including words the book's authors never saw, because fragments always work and single letters are the last resort.

A tokenizer is that shorthand book. It was built by counting what's common in a training corpus. Text that resembles the corpus compresses well, and unusual text doesn't.

Text type	How the shorthand copes
Common English	Mostly one symbol per word
Rare technical terms	Several fragments
Random IDs like `A91X`	Close to letter by letter

Why not use words or characters?

Unit	Vocabulary	Problem
Whole words	Huge, and never complete	`getCustomerById`, new product names and typos have no entry
Characters	Tiny, about 100	Each word takes many steps, which is slow and costly, and the model must relearn spelling
Subword tokens	Tens of thousands to a few hundred thousand	The practical compromise: any text can be encoded, and common words stay cheap

Sequence length matters because every token passes through the model, and attention cost grows with length (AI02 L2).

How byte-pair encoding works

Byte-pair encoding (BPE), applied to language modeling by Sennrich and colleagues, is the classic method. The training algorithm has one idea, repeated:

1. Start with every word split into characters.
2. Count all adjacent pairs across the corpus.
3. Merge the most frequent pair into a new symbol.
4. Repeat until the vocabulary reaches the size you want.

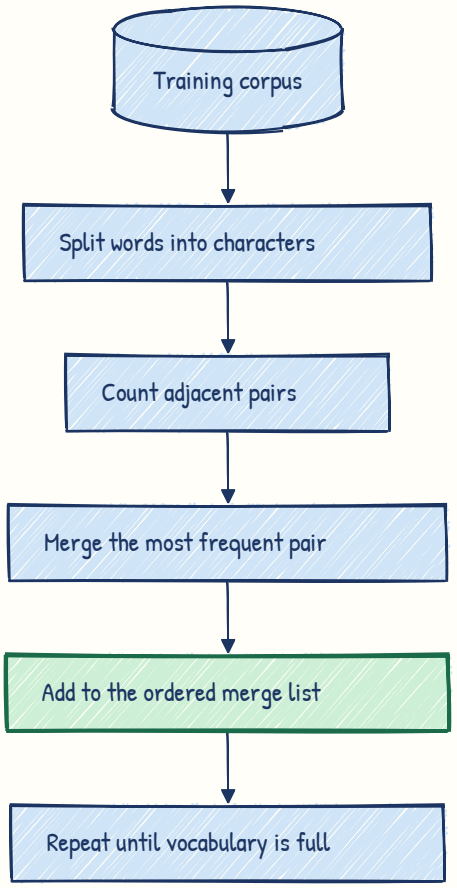


Figure 1. Training a BPE tokenizer. Each merge adds one entry to the vocabulary. The ordered list of merges is the tokenizer.

Encoding new text applies the merges in their learned order. Here's the core, from `code/ai02-llm/Bpe.java`:

```
static List<String> encode(String text, List<Merge> merges) {
    List<String> tokens = new ArrayList<>();
    for (String word : text.trim().split("\\s+")) {
        List<String> symbols = symbols(word);
        for (Merge merge : merges) {
            symbols = apply(symbols, merge);
        }
        tokens.addAll(symbols);
    }
    return tokens;
}
```

Real output, after 30 merges on a tiny support-themed corpus. The `_` marks the end of a word.

```
first merges:
  r + e
  d + _
  u + n
  re + f
  ref + un
  m + e
refund            1 tokens [refund_]
refunded          3 tokens [refun, de, d_]
refunding         6 tokens [refun, d, i, n, g, _]
payment           1 tokens [payment_]
getCustomerById  10 tokens [g, e, t, C, ustome, r, B, y, I, d_]
25 chars ->  6 tokens: the customer order refund
20 chars -> 21 tokens: {"id":"A91X","st":2}
```

Four lessons are visible in those lines:

1. **Frequent words become a single token.** `refund` and `payment` compress completely.
2. **Unseen words still work.** Nothing fails on `refunding`. The cost simply rises.
3. **Identifiers fragment.** `getCustomerById` takes 10 tokens here.
4. **JSON can cost more than the prose around it.** In this demo, 20 characters of JSON took 21 tokens, because the punctuation and IDs were unfamiliar. Here, 25 characters of familiar prose took 6.

Don't copy those exact numbers

This is a 30-merge toy tokenizer, trained on a few dozen words. Real ones have vastly larger vocabularies, and they handle common JSON punctuation far better. **The direction is what carries over: content unlike the training data costs more.** Measure your own payloads.

The tokenizer family

Method	Core idea	Known use
BPE	Merge frequent pairs	Widely used in GPT-style models
WordPiece	Greedy longest-match with learned pieces	BERT
SentencePiece / Unigram	Language-independent, works on raw text	Many multilingual models

What about Claude? Anthropic doesn't publish its tokenizer's internals, and they're out of scope for the exam. You don't need them, because the API counts for you.

From tokens to numbers

Tokenizers produce integer IDs, and the model looks up each ID in its embedding table (AI02 L2).

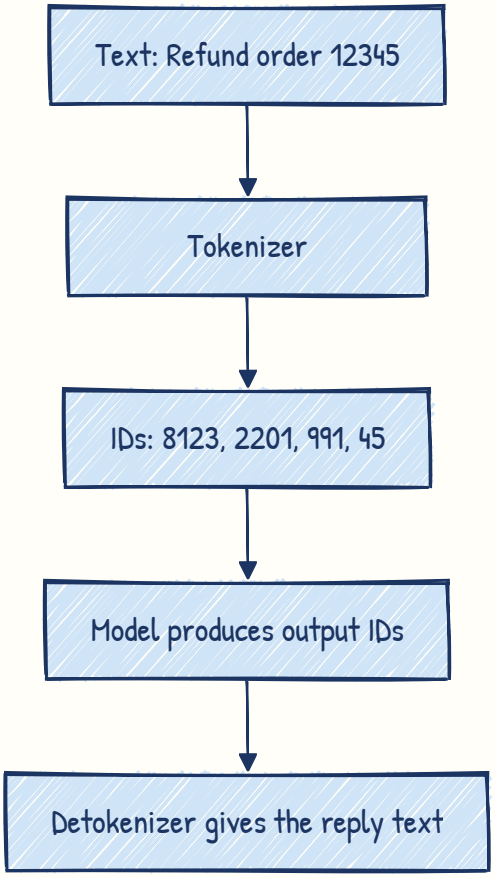


Figure 2. The path from your text to the model and back. The model only ever sees integer IDs. These IDs are made up for illustration.

Special tokens are reserved entries with structural meaning, such as start of text, end of text and padding. Chat systems also use internal markers separating roles, and on the Messages API you never handle them directly.

Tokens and the surprises they cause

Since the model sees tokens rather than letters, some well-known behaviors follow. Counting letters in a word, or reversing a string, is harder than it looks, because the model doesn't necessarily see each letter as a separate unit. **This is a design hint: use a tool or code for exact character work, rather than asking the model.**

Task	Better approach
Count characters	Java `String.length()`, via a tool if needed
Exact arithmetic on many digits	A calculator tool
Compare two IDs for equality	Code
Summarize or classify	The model

How Anthropic approaches it

Anthropic's contribution here is practical: making counts measurable.

Problem	What Anthropic provides
You can't compute Claude's tokens locally	A `count_tokens` endpoint, taking the same inputs as a real message, including system, tools, images and PDFs
Counts differ across models	It counts using the tokenizer of the `model` you pass, so you can compare by counting twice
Counting costs could discourage checking	It's free, but rate limited separately from message creation
Billing must be verifiable	Each response returns a `usage` object

Details worth remembering, all from that page:

→ The count is an **estimate**, and the actual figure may differ by a small amount.

→ It may include tokens Anthropic adds for system optimizations, and you aren't billed for those.

→ It returns an error for server tools other than the advisor tool, the MCP connector, and `url` or `file` document sources. For those requests, read `usage` on the response.

→ It doesn't use prompt caching logic.

> *Version-sensitive*
>
> The 30% figure and the endpoint's limits were checked on 4 November 2026. Recheck them when you depend on them.

The production architecture

This is a recommended reference design, not a specific company's system. The question: how do you stop oversized requests, without slowing every request?

Baseline: estimate from string length. Cheap, and it's wrong by an unknown amount. It fails on JSON, code and non-English text, and the first signal is a rejected request in production.

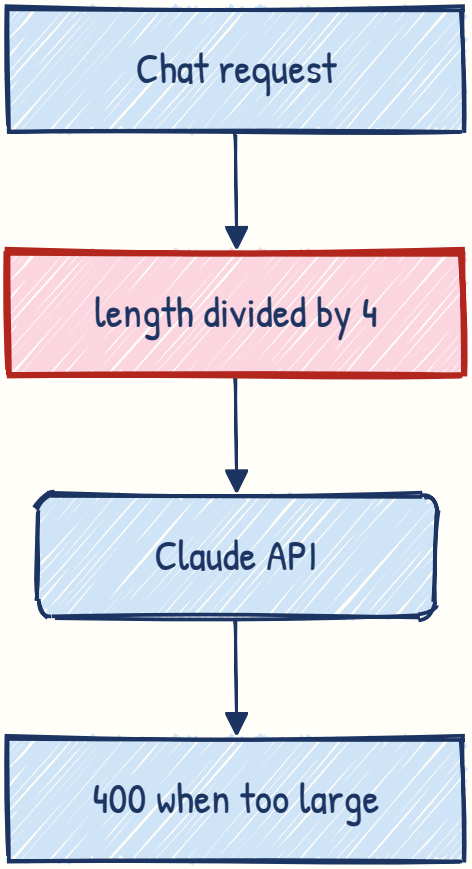


Figure 3. Baseline. A character-based guess decides what's sent, so requests with unusual content fail at the API.

Improved: count every request. Accurate. New problem: **it adds a network hop to every call, and consumes a separate rate limit.**

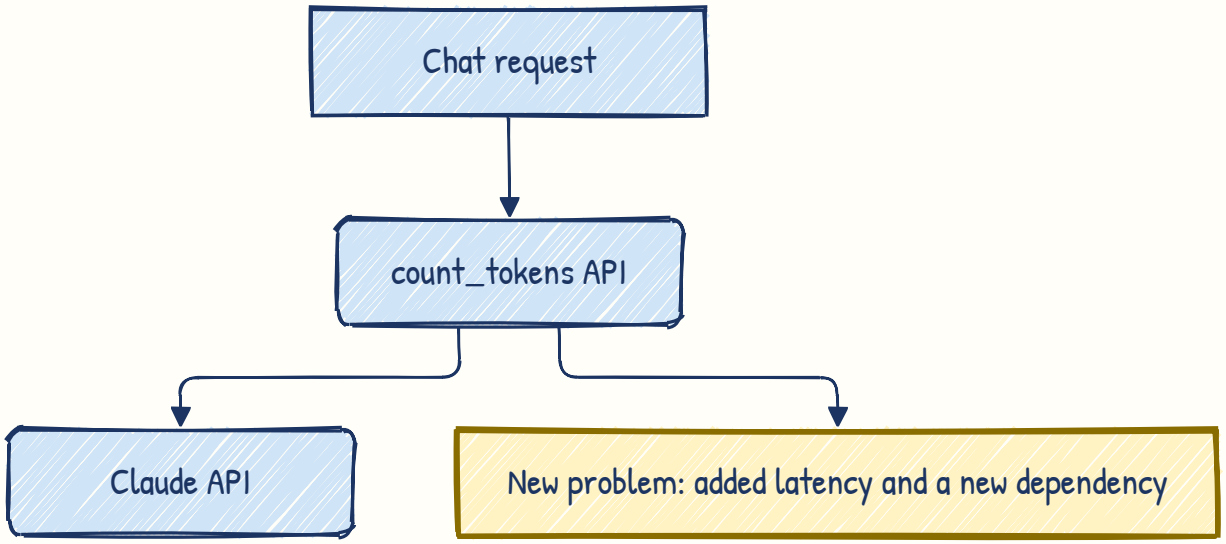


Figure 4. Improved. Counts are accurate, but each request now pays a round trip, and a counting outage could block traffic.

Production: estimate first, count when close. A rough local estimate clears clearly small requests. Only requests near the limit pay for an exact count, and actual `usage` calibrates the estimator.

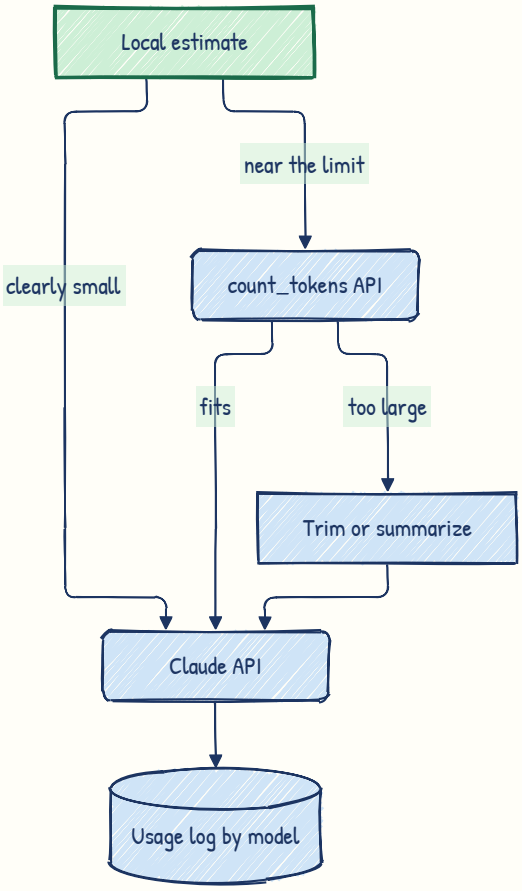


Figure 5. Production. Small requests skip the extra hop, and only those near the limit get an exact count. Periodically, the usage log recalibrates the estimator.

Component	Why it exists	How it communicates
Local estimate	Avoids a hop for most traffic	In-process, with a ratio per model and content type
`count_tokens`	Accuracy when it matters	Synchronous HTTP with a short timeout
Trim step	Fits oversized history	Application logic (AI03 L6)
Usage log	Real numbers for calibration and chargeback	Async write, keyed by model and tenant

Failures

Failure	Effect	Containment
Estimator ratio is stale after a model change	Near-limit requests get through and fail	Re-baseline ratios for each model as part of migration
`count_tokens` is down or rate limited	No exact count	Fail open with a conservative trim, rather than blocking
User pastes a huge document	Cost and latency spike	Enforce a size limit at the gateway, before the estimate
Attacker sends token-heavy junk	Inflated bill	Per-tenant token budgets (AI10 L5)

Java

This pre-flight check compiles against `anthropic-java` 3.2.0. It hasn't been run, since no API key is available here.

```
public Decision check(String system, String userMessage, long reservedForOutput) {
    long input = client.messages().countTokens(
            MessageCountTokensParams.builder()
                    .model(model)
                    .system(system)
                    .addUserMessage(userMessage)
                    .build())
            .inputTokens();
    return new Decision(input, input + reservedForOutput <= contextWindow);
}
```

`reservedForOutput` is the decision that matters. Input and output share the context window. If you fill the window with input, there's no room left for a reply. Gradle:

```
implementation("com.anthropic:anthropic-java:3.2.0")
```

Trade-offs

Method	Accuracy	Latency	Use it for
Character ratio	Rough	None	Fast path, capacity planning
`count_tokens`	Good estimate	A round trip	Near-limit decisions
`usage` on the response	Exact and billed	After the fact	Billing, calibration, alerts

When not to bother with pre-flight checks: if your inputs are small and bounded, such as classifying a short ticket, a gateway size limit is enough. Read `usage`, and move on.

Common mistakes

1. **Treating one token as one word.** Counts vary by content.
2. **Reusing budgets across models.** The tokenizer may differ. Recount when migrating.
3. **Forgetting the output.** The window must also hold the reply.
4. **Using another vendor's tokenizer.** It's counting for a different model.
5. **Dumping verbose JSON into prompts.** You pay for every key and brace, so send only the fields needed.
6. **Asking the model for exact character counts.** Use code.

What to say in an interview

Asked why JSON is costly, or why a migration changed costs, say: "Tokens depend on the tokenizer, and content unlike its training data fragments more. So I'd count against the target model before migrating, and I'd log real usage by model." Then mention what you'd do about it: send fewer fields.

Certification lens

How this lesson maps to CCAR-F

Token counting algorithms and tokenization specifics are explicitly out of scope. What's tested is the consequence: tool results "consume tokens disproportionately to their relevance", for example 40+ fields per order lookup when only 5 are relevant (TS 5.1).

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Customer Support Resolution Agent · Medium. Long support sessions hit context limits early. Logs show `lookup_order` returns over 40 fields per call, of which the agent uses about 5. What's the most effective first step?

A. Move to a model with a larger context window.

B. Summarize the whole conversation after every tool call.

C. Trim tool results to the relevant fields before they accumulate in context.

D. Limit each customer to three order lookups per session.

A **C.** This removes the waste at its source (TS 5.1). A postpones the problem and raises cost. B risks losing exact values in summaries. D harms customers with several orders.

Q Q2 · Troubleshooting · Medium. After migrating models, your 180,000-token prompts start failing a 200,000-token budget check, though the text is unchanged. Why, and what do you do?

A The newer tokenizer can produce more tokens for the same text, about 30% more on Claude 4.7 and later, per Anthropic's documentation. Recount against the target model, then update budgets or trim. Don't reuse the old counts.

Q Q3 · True or false · Easy. `count_tokens` returns the exact number that will be billed.

A **False.** The documentation calls it an estimate that can differ slightly. Billing comes from `usage` on the actual response.

Revision - Lesson 1

→ A token is the unit an LLM reads, writes and bills. It isn't a word.

→ Subword tokens are the compromise between huge word vocabularies and slow characters.

→ BPE repeatedly merges the most frequent adjacent pair. The ordered merges are the tokenizer.

→ Unseen text still encodes, at a higher token cost. Identifiers and unusual JSON fragment more.

→ Counts depend on the model. Claude 4.7+ produces about 30% more tokens than earlier models.

→ `count_tokens` is a free, separately rate-limited estimate. `usage` is what's billed.

→ Input and output share the context window, so reserve room for the reply.

→ Production: estimate locally, count near the limit, log real usage, and re-baseline on migration.

→ Use code rather than the model for exact character work.

Your notes